Software Architecture and Design Specification

Project: Jackfruit Project

Version: 1.0

Authors: Jackfruit Project Team

Date: 05-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Jackfruit Project Team	Initial SAD based on the SRS

Approvals

Role	Name	Signature/Date
Course Coordinator		
Project Team		

1. Introduction

1.1 Purpose

This document specifies the proposed software architecture and design of the Jackfruit Project. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS) into architectural components, interfaces, data responsibilities, security controls, and design flows.

1.2 Scope

The architecture covers the centralized web-based Blood Bank Management System described in the SRS, including donor registration and eligibility, blood collection, blood inventory, hospital/patient requests, request approval, blood-unit allocation and issue, notifications, reporting, user and role management, audit logging, and backup/recovery.

1.3 Audience

Developers, database designers, QA engineers, system administrators, blood-bank staff, hospital/requesting staff, course instructors, and assessment evaluators.

1.4 Definitions

- BBMS – Blood Bank Management System.
- RBAC – Role-Based Access Control.
- FEFO – First-Expire, First-Out.
- API – Application Programming Interface.
- Audit Log – Tamper-resistant record of important system actions.
- Blood Unit – A collected and uniquely identified unit of blood or blood component.

2. Document Overview

2.1 How to use this document

This document describes the proposed architecture and detailed design of the BBMS. It should be used together with the SRS to understand how requirements are mapped to components, interfaces, data stores, security controls, and system flows. UML diagrams are to be inserted in the indicated locations.

2.2 Related Documents

Software Requirements Specification (SRS), Test Plan/Software Test Plan, and Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Architecture goals:

- Provide clear separation between presentation, application/business logic, data access, and supporting services.
- Protect donor, patient, requester, authentication, and inventory information through role-controlled access.
- Maintain inventory consistency when units are collected, reserved, allocated, issued, discarded, or have their status changed.
- Provide end-to-end traceability through audit records.
- Support searchable records, reporting, notifications, and administrative monitoring.

Constraints derived from the SRS:

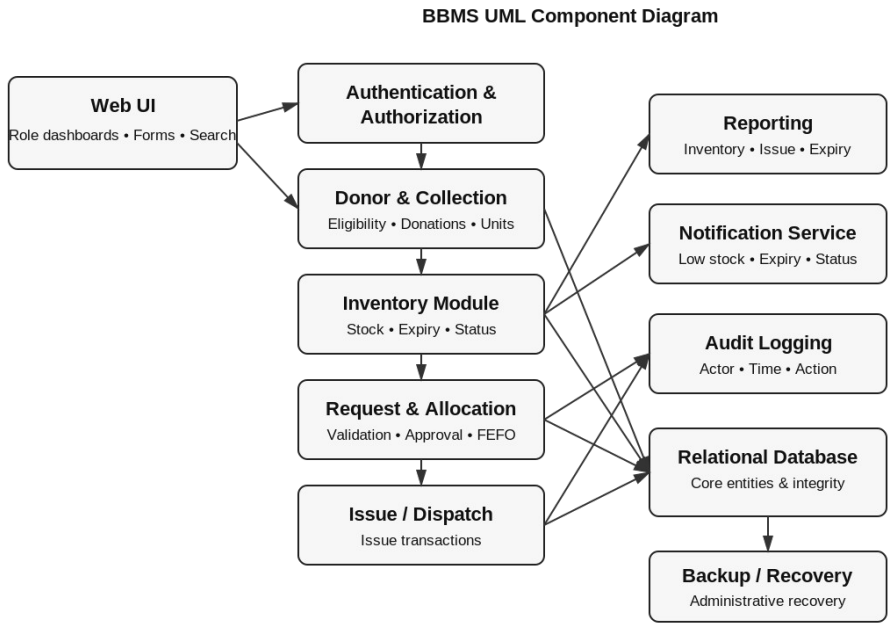
- The system is a centralized web-based information system.
- Core entities must use a relational database with integrity constraints.
- Client-server communication shall use HTTPS/TLS.
- Only authorized staff may change critical blood-unit status or issue records.
- Every blood unit must be uniquely identifiable and traceable.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns

Donor	Accurate registration and donation information.
Blood Bank Staff	Donor processing, collection, inventory accuracy, and request fulfillment.
Hospital/Requester	Submitting and tracking authorized blood requests.
Administrator	Users, roles, configuration, reports, and audit activities.
System/Database Administrator	Deployment, database, backups, recovery, and system health.
Developers / QA	Modularity, testability, correctness, security, and maintainability.

3.3 Component (UML) Diagram



The proposed logical components are Web UI, Authentication and Authorization, Donor and Collection, Inventory, Request and Allocation, Issue/Dispatch, Reporting, Notification, Audit Logging, Relational Database, and Backup/Recovery.

3.4 Component Descriptions

Web UI: Provides role-specific dashboards, forms, validation, searchable/filterable tables, status messages, and confirmation before critical actions.

Authentication & Authorization: Provides secure authentication, session/token controls, and RBAC enforcement.

Donor & Collection Module: Manages donor profiles, eligibility, donation history, collection records, and unique blood-unit creation.

Inventory Module: Maintains blood-unit group/component, availability, expiry, quarantine, discard, and stock information.

Request & Allocation Module: Creates and validates requests, manages approval/rejection/hold, and allocates eligible units using FEFO where applicable.

Issue/Dispatch Module: Records blood-unit issue transactions and ensures corresponding inventory updates.

Reporting Module: Generates inventory, donation, request, issue, expiry, and discard reports.

Notification Service: Generates configured alerts for low stock, upcoming expiry, and important request-status changes.

Audit Logging Service: Records actor, timestamp, action, and relevant record reference for critical and security-sensitive events.

Relational Database: Stores core entities and relationships while enforcing data integrity constraints.

Backup/Recovery: Supports backup, recovery, and administrative monitoring as required by the SRS.

3.5 Chosen Architecture Pattern and Rationale

Proposed pattern: Layered architecture. The SRS specifies a centralized web application consisting of a web-based user interface, application services, a relational database, authentication/authorization services, notification services, and reporting facilities. A layered structure is therefore proposed to provide separation of concerns, maintainability, and testability.

A microservices architecture is not required by the SRS and is not selected for the initial design because it would add deployment and operational complexity beyond the stated system scope.

3.6 Technology Stack & Data Stores

The SRS specifies a modern web browser, an application server, a relational DBMS, secure TCP/IP or Internet connectivity, and optional barcode/QR and notification services. It does not specify exact programming languages, frameworks, or a specific database product.

Proposed implementation choices, to be confirmed by the project team:

- Responsive web frontend.
- REST-style application services.

- Relational DBMS for persistent storage.
- Secure session/token authentication with RBAC.
- HTTPS/TLS for client-server communication.
- Optional email/SMS notification and barcode/QR integration.

3.7 Risks & Mitigations

Risk: Unauthorized access **— Mitigation:** Enforce RBAC, secure authentication, session controls, input validation, and audit logging.

Risk: Inventory inconsistency **— Mitigation:** Use transactional/atomic updates for collection, allocation, issue, discard, and status changes.

Risk: Expired or quarantined unit issued **— Mitigation:** Validate eligibility and availability before allocation and issue.

Risk: Data loss **— Mitigation:** Provide backup/recovery procedures and administrative monitoring.

Risk: Sensitive information exposure **— Mitigation:** Use HTTPS/TLS, secure password hashing, controlled access, and avoid sensitive information in logs.

3.8 Traceability to Requirements

Requirement	Architectural Component	Design Responsibility
BBMS-F-001 to F-003	Donor & Collection	Registration, eligibility, and donation history.
BBMS-F-004 to F-008	Collection + Inventory	Unique units, unit details, stock, expiry, quarantine, release, and discard.
BBMS-F-009 to F-011	Request & Allocation	Request creation, validation, and decisions.
BBMS-F-012 to F-015	Allocation + Issue + Inventory	FEFO allocation, issue validation, traceability, and atomic updates.
BBMS-F-017 to F-018	Reporting + Notification	Reports and operational alerts.
BBMS-F-019 to F-020	Authentication/Authorization + Audit	User/role management and audit events.
BBMS-NF-001 to NF-005	Cross-cutting architecture	Performance, availability, security, retention, usability.
BBMS-SR-001 to SR-005	Security architecture	RBAC, authentication, password protection, input validation, and security

auditing.

3.9 Security Architecture

- RBAC restricts users to functions and records permitted for their role.
- Secure authentication and inactive-session expiration shall follow configured policy.
- Passwords shall use a strong one-way hashing mechanism and shall never be stored in plaintext.
- User input shall be validated and sanitized; database operations shall use parameterized operations.
- All client-server communication shall use HTTPS/TLS.
- Audit records shall capture failed logins, role changes, blood-unit status changes, approvals, issue transactions, and other configured security events.

The SRS does not mandate a particular threat-modeling framework. The final threat model should therefore be completed during architecture review and mapped to the implemented security controls.

4. Design

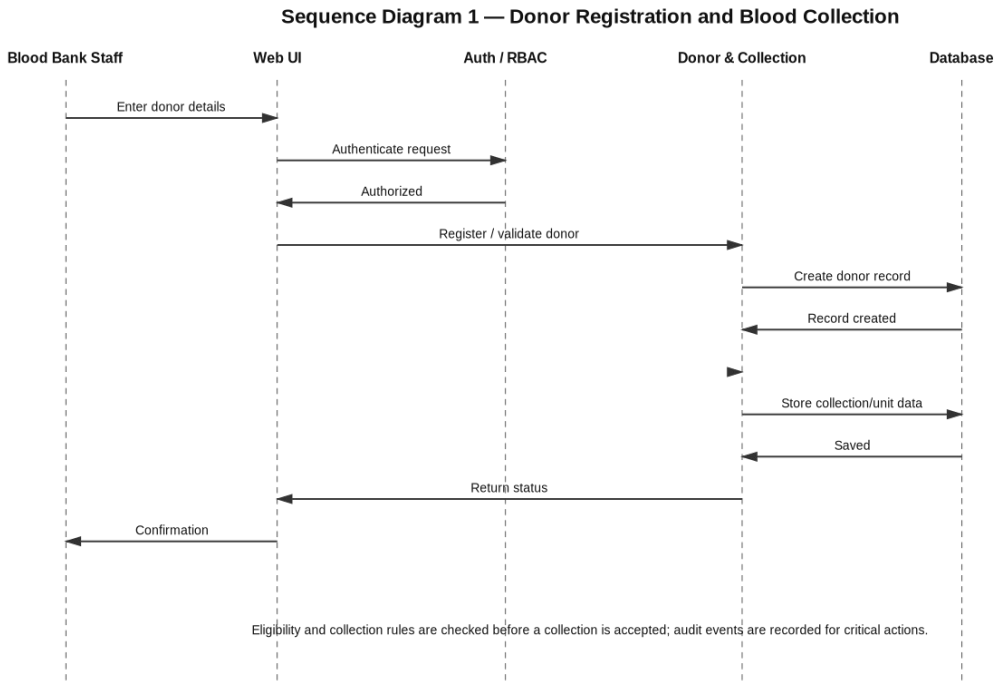
4.1 Design Overview

The proposed design separates presentation, authentication/authorization, domain modules, data access, and supporting services. A request enters through the web interface, is checked for authorization and business-rule validity, is processed by the relevant module, updates the relational database transactionally where required, records audit events, and returns a controlled response.

4.2 UML Sequence Diagrams

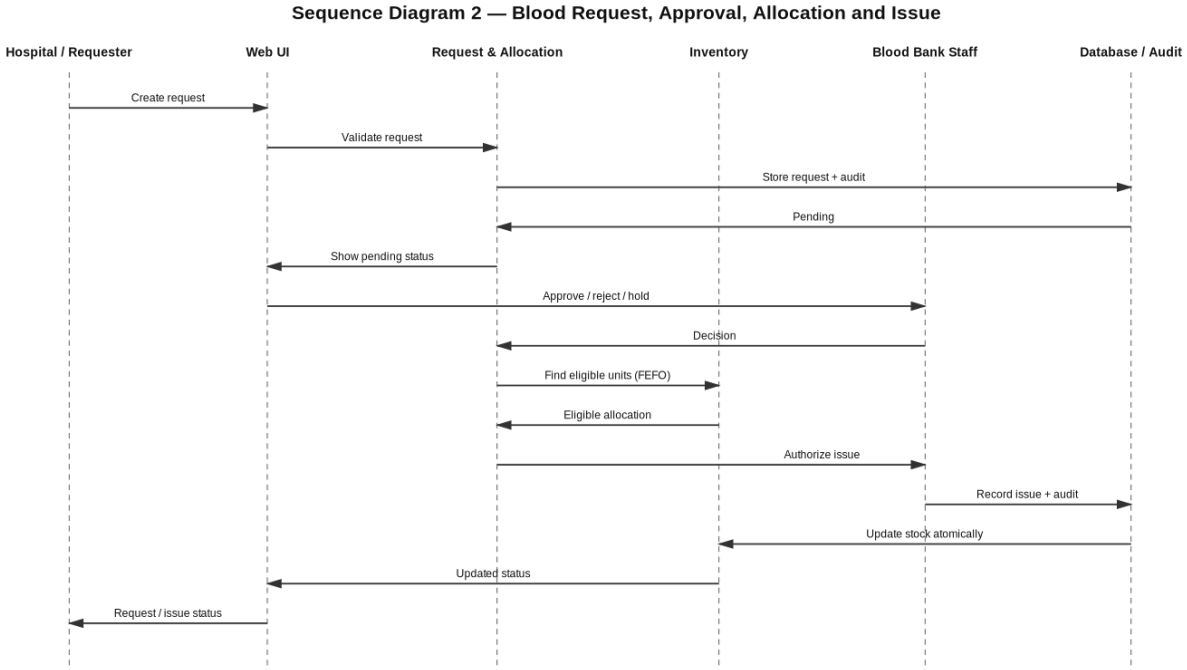
At least two project-specific sequence diagrams are required.

Sequence Diagram 1 – Donor Registration and Blood Collection



Suggested flow: Blood Bank Staff → Web UI → Authentication/Authorization → Donor & Collection Module → Relational Database → Inventory Module → Audit Logging.

Sequence Diagram 2 – Blood Request, Approval, Allocation and Issue



Suggested flow: Hospital/Requester → Web UI → Request & Allocation Module → Inventory Module → Blood Bank Staff/Administrator approval → Issue/Dispatch Module → Database → Audit Logging → Notification Service.

4.3 API Design

The SRS does not define concrete endpoint names. The following are proposed logical interfaces to illustrate component design; final endpoint paths and payload schemas should be confirmed during implementation.

Endpoint	Purpose	Request	Response / Errors
POST /api/donors	Create donor record	Donor details and eligibility information	Created donor record / validation error
POST /api/blood-requests	Create blood request	Requester, patient, group, component, quantity, priority, required date	Request ID and status / validation error
POST /api/requests/{id}/approve	Approve an authorized request	Request ID and decision information	Updated request status / authorization or validation error
POST /api/issues	Record blood-unit issue	Request ID, unit ID(s), quantity, issuing staff	Issue transaction / invalid-unit or transaction error

4.4 Error Handling, Logging & Monitoring

- Return clear validation errors without exposing sensitive information.
- Prevent issue of expired, quarantined, already issued, or unavailable units.
- Use atomic updates so failed collection, allocation, issue, discard, or status-change operations do not leave partial inventory changes.
- Record audit events for critical and security-sensitive actions.
- Monitor performance, availability, transaction failures, low stock, upcoming expiry, and important request-status changes as applicable.

4.5 UX Design

The SRS requires a responsive web application with role-specific dashboards. Forms shall validate mandatory fields, provide clear status messages, support searchable/filterable tables, confirm critical actions, and provide accessible navigation.

4.6 Open Issues & Next Steps

- Confirm the implementation technology stack and specific relational DBMS.
- Create and review the UML component diagram.
- Create the two required sequence diagrams and insert them into the final SAD.

- Finalize API paths, payload schemas, authentication mechanism, and deployment topology.
- Complete the threat model and map threats to controls.
- Review the SAD against the final implementation and RTM before submission.

5. Appendices

5.1 Glossary

- BBMS
- RBAC
- FEFO
- API
- Audit Log
- Blood Unit

5.2 References

Software Requirements Specification (SRS), Jackfruit Project.

SAD template supplied for the project.

5.3 Tools

Diagramming tools may include draw.io/diagrams.net or PlantUML. API documentation tooling may be selected during implementation. Specific tools are to be confirmed by the project team.